

코데이 아키텍처 최종 점검 (2026-09-04)

성능 점검(9절) 직전 단계에서 진행한 구조 점검. 실제 소스코드(~/AndroidStudioProjects/KkokDay)를 직접 읽고 확인한 내용이며, 추측이나 일반론이 아니라 이 프로젝트에 실제로 있는 코드를 근거로 작성했다.

결론 먼저

“예전엔 View/ViewModel/Repository로 패키지를 나뉘는데, 이번엔 처음 보는 구조라 이게 맞나?” — **맞다. 오히려 예전 방식보다 실무에 훨씬 가깝다.**

지금 구조는 “**data는 도메인별, ui는 기능별**”로 나누는 방식이다:

```
data/
  auth/      (AuthRepository + AuthRepositoryImpl + ...)
  place/     (FavoriteRepository + FavoriteRepositoryImpl, RecentPlaceRepository + Impl, ...)
  review/
  course/
  user/
  ...
di/          (Hilt 모듈 - 인터페이스 ↔ 구현체 바인딩)
navigation/  (라우트 정의 + NavHost)
ui/
  category/  (CategoryPlacesScreen + CategoryPlacesUiState + CategoryPlacesViewModel + 컴포넌트들)
  favorite/  (FavoritesScreen + FavoritesUiState + FavoritesViewModel)
  course/
  ...
```

예전 방식(`view/`, `viewmodel/`, `repository/` 로 최상위를 나누는 것)은 **레이어 기준(package by layer)** 패키징이다. 지금 방식은 **기능/도메인 기준(package by feature)** 패키징이다. 실무에서는 프로젝트가 커질수록 레이어 기준이 오히려 불편해진다 — 화면 하나를 고치려면 `view/`, `viewmodel/`, `di/` 세 군데를 넘나들어야 하고, 관련 없는 기능들이 같은 폴더(`viewmodel/` 전체)에 뒤섞인다. 구글 공식 아키텍처 가이드도 앱 규모가 커지면 feature 기준 패키징을 권장한다. **코데이는 이미 그 방식으로 짜여 있다** — MVVM이 안 지켜진 게 아니라, MVVM을 “어떻게 폴더로 담을지”에 대한 더 발전된 답을 쓰고 있는 것.

각 기능 폴더 안에서 `Screen` + `UiState` + `ViewModel` 세 파일이 세트로 붙어다니는 것도 의도된 구조다(아래 3절 참고). 즉 “MVVM이 안 보인다”가 아니라 “MVVM 단위가 클래스가 아니라 기능 폴더”인 것.

1. MVVM 적용 상태 — 정석대로 되어 있음

`CategoryPlacesViewModel`, `FavoritesViewModel` 등 모든 `ViewModel`이 동일한 패턴을 따른다:

```
private val _uiState = MutableStateFlow(FavoritesUiState())
val uiState: StateFlow<FavoritesUiState> = _uiState.asStateFlow()
```

- **State**는 **private mutable + public read-only 노출**. 외부(Compose 쪽)에서 상태를 직접 바꿀 수 없고 `ViewModel`의 함수를 통해서만 바뀐다 — MVVM의 핵심인 “View는 `ViewModel`의 상태를 구독만 한다”가 코드 레벨로 강제되어 있다.
- **UiState**는 전부 불변 `data class`. `state.copy(...)` 로만 갱신 — 상태 변경 추적이 쉽고 리컴포지션 최적화에도 유리하다.
- **Repository** 인터페이스만 생성자로 주입받는다 (`FirebaseFirestore` 를 `ViewModel`이 직접 들고 있지 않음). `View` → `ViewModel` → `Repository`(인터페이스) → `RepositoryImpl`(`Firebase`) 의존 방향이 항상 한쪽으로만 흐른다.

2. SOLID 원칙 — 대부분 잘 지켜져 있고, 어긴 지점도 꼭 집을 수 있음

원칙	지켜진 예	비고/예외
SRP (단일 책임)	FavoriteRepository 는 즐겨찾기만, PopularityScore.kt 는 인기순 점수 계산만, PlaceDocId.kt 는 문서 ID 규칙만 담당	CategoryPlacesViewModel 이 검색+정렬+탭+즐거찾기+코스탐기+평점enrich까지 떠안아 380줄 가까이 됨 — 아래 5절에서 상세
OCP (개방-폐쇄)	KkokDayRoute.bottomTabRoute() 에 새 탭을 추가해도 navigateToBottomTab 로직은 안 건드려도 되게 설계(코드 주석에도 명시)	—
LSP (리스코프 치환)	모든 Repository가 인터페이스 하나에 구현체 하나 — 테스트 용 Fake 구현체로 치환해도 ViewModel 쪽 코드는 전혀 안 바뀜	실제로 테스트 코드가 없어서 “치환 가능하다”가 실전 검증은 안 된 상태 (6절 참고)
ISP (인터페이스 분리)	FavoriteRepository 는 메서드 4개짜리 작은 인터페이스. “Firestore 관련 기능 전부”를 한 인터페이스에 몰아넣지 않고 도메인별로 쪼갬	—
DIP (의존성 역전)	di/PlaceModule.kt 등에서 @Binds abstract fun bindFavoriteRepository(impl: FavoriteRepositoryImpl): FavoriteRepository 패턴으로 ViewModel은 인터페이스만 알고 Firebase 구현은 모름	SignupViewModel.kt:136 에서 signUpError is FirebaseAuthUserCollisionException 으로 Firestore 예외 타입을 ViewModel이 직접 검사 — Firebase라는 구체적인 프레임워크 디테일이 화면 레이어까지 새어 올라온 유일한 지점

DIP 예외 하나는 구체적으로 짚어두면: 지금은 AuthRepository.signUp() 이 Result<Unit> 을 돌려주고, “이미 가입된 이메일이라 실패했다”를 ViewModel이 Firebase 예외 클래스를 보고 판단한다. 더 나은 형태는 AuthRepository 가 Result<SignUpOutcome> 처럼 앱 자체 타입(sealed class)을 돌려주는 것 — 그러면 ViewModel은 Firebase를 아예 몰라도 된다. 지금 규모에서 심각한 문제는 아니지만, “레이어 경계가 어디서 새는지” 찾는 연습으로는 딱 좋은 사례다.

3. Jetpack Compose — Route/Screen 분리 패턴이 이미 적용되어 있음 (이게 아마 낯설게 느껴진 부분)

ui/favorite/FavoritesScreen.kt 안에 함수가 이렇게 나뉘어 있다:

```
@Composable
fun FavoritesRoute(...) // public - hiltViewModel(), collectAsStateWithLifecycle()

@Composable
private fun FavoritesScreenContent(...) // private - UiState와 콜백 람다만 받는 순수 UI

@Composable
private fun EmptyFavorites(...) // private - 더 작은 조각

@Composable
private fun FavoritesScreenPreview(...) // @Preview 전용
```

이건 구글이 Compose 공식 아키텍처 가이드에서 권장하는 “Route(상태 있는 진입점) vs Screen(상태 없는 순수 UI)” 분리 패턴이다. 예전 View/ViewModel/Repository 구조에는 없던 개념이라 낯설게 느껴졌을 텐데, 정확히는 “View” 하나가 두 개의 Composable로 쪼개진 것이다:

- XxxRoute = 예전의 “Activity/Fragment가 ViewModel을 붙잡는 부분”
- XxxScreenContent = 예전의 “레이아웃 XML”에 해당하되, 상태를 직접 안 갖고 파라미터로만 받아서 재사용·미리보기(Preview)·테스트가 훨씬 쉬워진 버전

이 분리 덕분에 @Preview composable(FavoritesScreenPreview)이 실제 ViewModel/Firebase 없이도 즉시 렌더링된다 — Android Studio에서 화면을 확인할 때 앱을 매번 빌드해서 로그인부터 안 해도 된다는 뜻.

그 외 Compose/최신 기술 체크리스트:

- StateFlow + collectAsStateWithLifecycle() : collectAsState() 가 아니라 라이프사이클 인지 버전을 씀 — 화면이 백그라운드일 때 불필요한 리컴포지션을 막는, 실무에서 실제로 신경 쓰는 디테일.
- SavedStateHandle : CategoryPlacesViewModel 이 네비게이션 인자(categoryGroupCode)를 SavedStateHandle 로 받음 — 프로세스가 죽었다 복원돼도 인자가 유지되는 최신 방식.
- Hilt + hiltViewModel() : DI 프레임워크가 Compose Navigation과 통합되어 있음.
- callbackFlow (투표 실시간 리스너), Flow 기반 이벤트 스트림(CategorySearchEvent.Progress / Failed) — 콜백 기반 Firebase API를 코루틴 Flow로 감싸는 정석적인 방법.
- 낙관적 업데이트(Optimistic UI): onFavoriteToggleClick 이 서버 응답 기다리지 않고 먼저 하트를 바꾸고, 실패하면 롤백 — 실무 프로덕션 앱에서 자주 쓰는 UX 패턴을 직접 구현해본 것.

4. 몰랐을 수도 있는데 알아두면 좋은 것들

- **Delegate(위임) 패턴으로 ViewModel 간 로직 공유** — `ui/course/CourseSaveDelegate.kt` . “코스에 담기” 흐름을 `FavoritesViewModel` 과 `CategoryPlacesViewModel` 둘 다 써야 하는데, 상속으로 공유 base ViewModel을 만들기 애매한 상황(Hilt ViewModel은 다중 상속 구조를 만들기 까다로움)에서, 별도 클래스에 로직을 담고 각 ViewModel이 생성자에서 만들어 위임하는 방식을 썼다. “**상속보다 조합(favor composition over inheritance)**”이라는 OOP 원칙을 실전에서 쓴 사례 — 이 패턴 이름과 원리를 알아두면 다른 프로젝트에서도 바로 응용 가능하다.
- **베이지안 가중평균 (Bayesian average)** — `PopularityScore.kt` . IMDB/Reddit 등이 “리뷰 1개 5점”이 “리뷰 50개 평균 4.7점”을 이기지 못하게 하려고 쓰는 랭킹 알고리즘을 직접 구현해봤다. 이름과 공식($(v/(v+m)) * R + (m/(v+m)) * C$)을 기억해두면 면접에서 “정렬/랭킹 로직 설계해본 경험” 질문에 바로 쓸 수 있는 소재다.
- **NoSQL에서 UNIQUE 제약 흉내내기** — `NicknameRepository` + Firestore 트랜잭션. 관계형 DB의 UNIQUE 제약이 없는 Firestore에서, “닉네임 예약 문서”를 트랜잭션으로 원자적으로 생성해 중복 가입을 막는 방식(계획서 5-3절). NoSQL 모델링에서 자주 나오는 실전 트릭이다.
- **서버 측 집계(Fan-out on write)** — Cloud Functions의 `recalculatePlaceRating` 트리거. 리뷰가 쓰일 때마다 `places/{placeDocId}` 문서의 `avgRating / reviewCount` 를 서버에서 재계산해두는 이유는, 클라이언트가 매번 리뷰 전체를 읽지 않고 요약 문서 하나만 읽게 하기 위함 — Firestore/NoSQL에서 매우 흔한 “비정규화 + 트리거로 집계” 패턴.
- **컬렉션마다 다른 보안 규칙 설계** — 코스(`courses`)는 읽기 공개/쓰기는 소유자만, 투표(`votes`)는 로그인 없이도 읽기·쓰기 둘 다 허용. “인증”과 “인가”를 데이터 컬렉션 단위로 다르게 설계해본 경험은 백엔드 보안 모델링 이해도를 보여주기 좋은 소재다.

5. 개선하면 더 좋을 점 (급하지 않음 — 학습 관점의 다음 단계)

1. **도메인(UseCase) 계층이 없다.** 지금은 “Data 계층(Repository) → Presentation 계층(ViewModel)” 2계층이고, 검색 보안 로직·카테고리 분류 같은 비즈니스 로직이 ViewModel에 직접 있다. “Clean Architecture”에서 말하는 Data/Domain/Presentation 3계층 중 Domain이 빠진 형태다. 다만 이게 “틀렸다”는 아니다 — 지금 규모(공유되는 비즈니스 로직이 많지 않은 개인 프로젝트)에서는 실무에서도 흔히 UseCase 계층을 생략한다. 다만 **면접에서 “왜 UseCase를 안 뒀나/뒀다면 뭐가 바뀌나”는 물어볼 만한 질문**이니, Clean Architecture의 3계층 개념 자체는 설명할 수 있게 알아두면 좋다.
2. **CategoryPlacesViewModel 이 책임이 많다** (검색/정렬/탭 상태/즐거찾기/코스담기/평점 캐싱을 한 클래스가 처리, ~380줄). 이미 `CourseSaveDelegate` 로 로직을 분리해본 경험이 있으니, 같은 방식으로 “검색 보안(augment) 로직”만 별도 클래스로 뽑아내는 리팩터링을 셀프 스터디 삼아 해보면 좋은 연습이 된다.
3. **DIP가 새는 지점 1곳** — `SignupViewModel` 이 `FirebaseAuthUserCollisionException` 을 직접 검사(2절 참고). `AuthRepository` 가 sealed class 결과 타입을 돌려주도록 바꾸면 해결된다.
4. **자동화된 테스트가 없다.** `ExampleUnitTest / ExampleInstrumentedTest` 보일러플레이트만 있고 실제 테스트는 0개. 반가운 소식은 — 지금 구조(인터페이스 기반 Repository, 생성자 주입, 순수 함수로 뽑힌 `PopularityScore / PlaceDocId / WalkingTime`)가 **이미 테스트하기 쉬운 형태로 짜여 있다는 것**. `PopularityScore.kt` 나 `WalkingTime` 같은 순수 함수부터 유닛 테스트를 붙여보는 게 가장 쉬운 시작점이고, Fake Repository를 만들어 ViewModel 테스트를 붙이는 것도 지금 구조라면 바로 가능하다.
5. **사소한 정리 포인트** — `runAuthCatching` (공통 에러 래핑 함수)이 `data/auth` 패키지에 있지만 실제로는 12개 Repository 구현체가 전부 가져다 쓴다. 이름/위치가 실제 쓰임(전 도메인 공통 유틸)과 안 맞는 경우라, `data/common` 같은 패키지로 옮기면 더 자연스럽다. 기능상 문제는 전혀 없다.

요약

기술 스택(Hilt, Compose BOM 최신, Coroutines/Flow, Retrofit+kotlinx.serialization, Coil, DataStore)과 패턴(패키지-by-feature, Route/Screen 분리, Repository+DI, Result 기반 에러 처리, Delegate 조합)이 실무 수준으로 꽤 잘 맞춰져 있다. “예전 구조랑 달라서 불안하다”는 감각은 틀린 신호가 아니라 — **정확히 실무 구조로 넘어왔다는 신호**로 받아들이면 된다. 공부할 때 걱정 없이 이 코드를 “이게 표준적인 형태다”라는 전제로 보고 익혀도 된다. 위 5절 개선점들은 “잘못됐다”가 아니라 “다음 단계로 알아두면 좋은 것들” 정도로 취급하면 된다.